LeetCode Practice

18 Easy: Trees & BST

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

18
EASY

Patterns (in order)

1. Trees & BST (18)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

18 questions | tap a title to jump to it

Easy questions (18)		Go >
Trees & BST (18)		**Go >**
E	1. Same Tree	LC 100
E	2. Symmetric Tree	LC 101
E	3. Maximum Depth of Binary Tree	LC 104
E	4. Convert Sorted Array to Binary Search Tree	LC 108
E	5. Balanced Binary Tree	LC 110
E	6. Path Sum	LC 112
E	7. Invert Binary Tree	LC 226
E	8. Closest Binary Search Tree Value	LC 270
E	9. Sum of Left Leaves	LC 404
E	10. Find Mode in Binary Search Tree	LC 501
E	11. Minimum Absolute Difference in BST	LC 530
E	12. Diameter of Binary Tree	LC 543
E	13. Subtree of Another Tree	LC 572
E	14. Average of Levels in Binary Tree	LC 637
E	15. Two Sum IV - Input is a BST	LC 653
E	16. Search in a Binary Search Tree	LC 700
E	17. Minimum Distance Between BST Nodes	LC 783
E	18. Range Sum of BST	LC 938

Easy questions (18)

EASY | TOPIC 1 OF 1

Trees & BST

18 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Same Tree LC 100
2. Symmetric Tree LC 101
3. Maximum Depth of Binary Tree LC 104
4. Convert Sorted Array to Binary Search Tree LC 108
5. Balanced Binary Tree LC 110
6. Path Sum LC 112
7. Invert Binary Tree LC 226
8. Closest Binary Search Tree Value LC 270
9. Sum of Left Leaves LC 404
10. Find Mode in Binary Search Tree LC 501
11. Minimum Absolute Difference in BST LC 530
12. Diameter of Binary Tree LC 543
13. Subtree of Another Tree LC 572
14. Average of Levels in Binary Tree LC 637
15. Two Sum IV - Input is a BST LC 653
16. Search in a Binary Search Tree LC 700
17. Minimum Distance Between BST Nodes LC 783
18. Range Sum of BST LC 938

EASY | Trees & BST | LeetCode #100 | 1 of 18 | 1/18

Same Tree

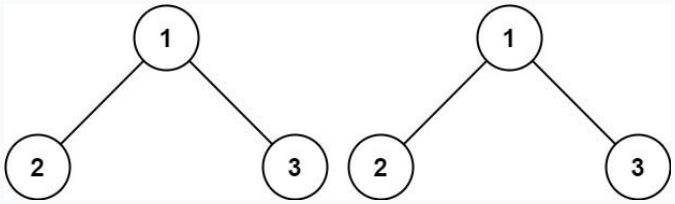
QUESTION

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

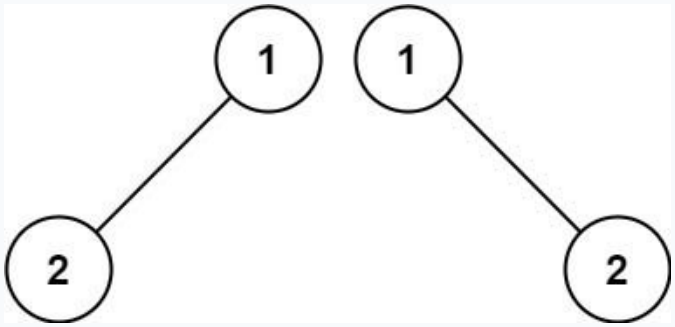
Example 1

Input	`p = [1,2,3], q = [1,2,3]`
Output	`true`



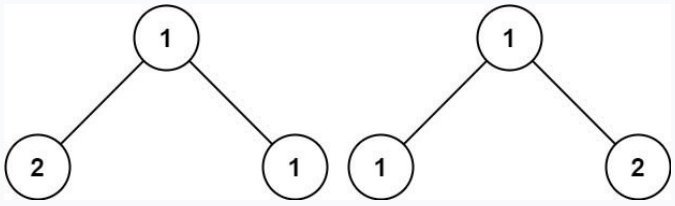
Example 2

Input	`p = [1,2], q = [1,null,2]`
Output	`false`



Example 3

Input	`p = [1,2,1], q = [1,1,2]`
Output	`false`



CONSTRAINTS

- The number of nodes in both trees is in the range [0, 100].
- 10^4 <= Node.val <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Two binary trees are the same if they're structurally identical AND every

corresponding node has the same value. Both null trees are the same. Right?

One null and one non-null at the same position -> not the same."

"p=[1,2,3], q=[1,2,3] -> true ok

p=[1,2], q=[1,null,2] -> false (structure differs) ok

p=[1,2,1], q=[1,1,2] -> false (values differ at depth 1) ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
list_a = pre-order of tree a (node, then left, then
    right, with markers for empty)
list_b = the same for b
return list_a == list_b
```

TIME	SPACE
O(n + m)	O(n + m)

WHY IT IS SLOW

Always visits and stores both trees completely. Walking both trees together and stopping at the first difference is O(min(n, m)) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive DFS comparing simultaneously — O(n). Walk-through:"

`def same(p, q):`

If p is None and q is None: return True (both ended).

If p is None or q is None: return False (one ended, not the other).

If p.val != q.val: return False (values differ).

`return same(p.left, q.left) AND same(p.right, q.right).`

Call same(p, q).

WHY short-circuit: if p.val != q.val, no need to recurse further.

WHY recurse both sides: subtrees must match in structure AND values.

HOW answer builds: base cases return True/False; each call ANDs left and right sub-answers.

A False from any node propagates up and short-circuits the AND chain.

Time: O(n). Space: O(h) call stack.

PHASE 4 CLEAN CODE

```python
class _100_SameTree:
    def isSameTree(self, p: Optional['TreeNode'], q: Optio
nal['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and same(p.left, q.left)
 and same(p.right, q.right)
        return same(p, q)
```

PHASE 5 TEST & DEBUG

"Let me trace isSameTree on [1,2,3] vs [1,2,3] and [1,2] vs [1,null,2]."

Case 1 — identical trees:

same(p=1,q=1): vals 1==1 ok

same(p.left=2, q.left=2): vals 2==2 ->
same(None,None)=T × 2 -> True

same(p.right=3, q.right=3): vals 3==3 ->
same(None,None)=T × 2 -> True

Both sides True -> True ok

Case 2 — structural diff [1,2] vs [1,null,2]:

same(p=1, q=1): vals 1==1 -> same(p.left=2,
q.left=None):

p is not None but q is None -> return False immediately

Result: False ok

Edge: both None -> True. One None one not -> False.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
visit every node once	call stack where h is tree height

Worst case O(n) for a skewed tree, O(log n) for balanced.

Iterative alternative: BFS with a queue of node pairs — O(n) time, O(n) space.

Same complexity but avoids recursion depth limit for very deep trees.

Given more time I'd ask: what if we only care about structural identity (not values)?

Same logic — just remove the val check.

IN YOUR APP | Expose | Editorial | Community

EASY | Trees & BST | LeetCode #101 | 2 of 18 | 2/18

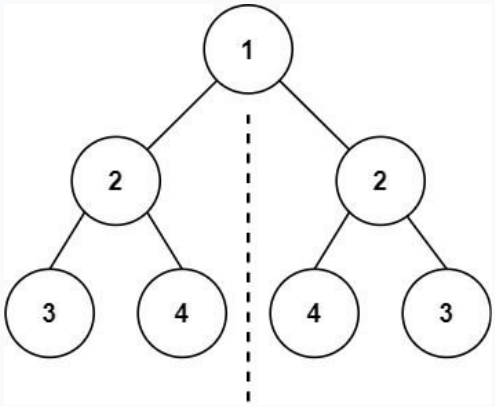
Symmetric Tree

QUESTION

Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

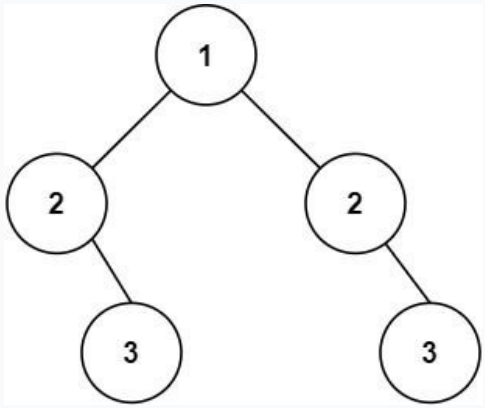
Example 1

Input	`root = [1,2,2,3,4,4,3]`
Output	`true`



Example 2

Input	`root = [1,2,2,null,3,null,3]`
Output	`false`



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 1000].
- 100 <= Node.val <= 100
- Follow up: Could you solve it both recursively and iteratively?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A tree is symmetric if its left and right subtrees are mirror images of each other.

Mirror means: at each level, left child of left subtree matches right child of right subtree,

and right child of left subtree matches left child of right subtree. Right?

PHASE 1 CLARIFY

A single node is always symmetric."

"[1,2,2,3,4,4,3]: left subtree [2,3,4] mirrors right [2,4,3] -> true ok

[1,2,2,null,3,null,3]: left has right child 3, right has right child 3 — not mirrored -> false ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
left = pre-order of the left subtree (empty markers
    included)
right = mirrored pre-order of the right subtree
    (right child first)
return left == right
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Builds two full lists even if the top two children already differ. A recursive mirror check (left.left with right.right, left.right with right.left) stops at the first mismatch and uses O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Recursive DFS comparing the tree to itself — left vs right mirrored.

Define an inner helper mirror(p, q):

Base: if either is None, both must be None.

Recursive: values match AND mirror(p.left, q.right) AND mirror(p.right, q.left).

Note the cross-comparison: left's left right's right, left's right right's left."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

Key insight: the symmetric check decomposes into mirror(p.left, q.right) AND mirror(p.right, q.left) — two subtrees checked together, each node visited once.

PHASE 4 CLEAN CODE

```
class _101_SymmetricTree:
    def isSymmetric(self, root: Optional['TreeNode']) -> b
ool:
        def mirror(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and mirror(p.left, q.rig
ht) and mirror(p.right, q.left)
        return mirror(root, root)
```

PHASE 5 TEST & DEBUG

"Let me trace mirror(left, right) on the symmetric and asymmetric examples."

root=[1,2,2,3,4,4,3]:

`mirror(2,2): values match ok`

outer pair: mirror(3,3) -> values match, then mirror(None,None) twice -> True ok

inner pair: mirror(4,4) -> values match, then mirror(None,None) twice -> True ok

Both outer and inner comparisons return True, so root is symmetric ok

root=[1,2,2,null,3,null,3]:

`mirror(left=2, right=2): values match, but now compare outer children:`

`mirror(None, 3) -> one side missing, so return False immediately`

That False bubbles back to the root, so answer = False ok

WHY this works: every recursive call compares nodes that should mirror each other across the center.

If values differ or one side is missing, symmetry breaks at that exact level.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** for the call stack

Follow-up asks for iterative solution: use a queue of pairs.

Enqueue (root.left, root.right). Each step dequeue (p,q):

check p==q==None (skip), p or q None (false), vals differ (false).

Enqueue (p.left,q.right) and (p.right,q.left). O(n) time, O(n) space.

The cross-enqueue order is the key — same insight as the recursive version.

Given more time I'd ask: how does this differ from Same Tree (#100)?

Same Tree checks identical structure; Symmetric Tree checks mirrored structure —

the only difference is the cross-comparison in the recursive call.

EASY | Trees & BST | LeetCode #104 | 3 of 18 | 3/18

Maximum Depth of Binary Tree

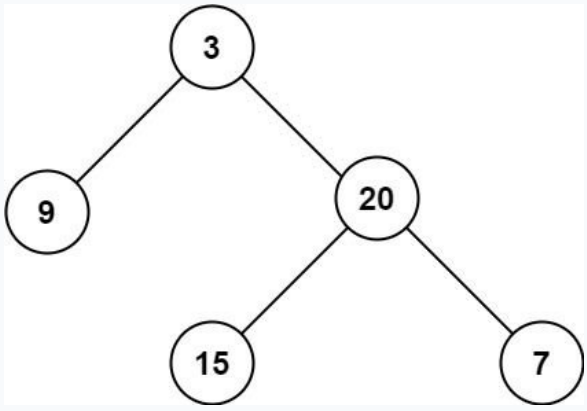
QUESTION

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`3`



Example 2

Input	`root = [1,null,2]`
Output	`2`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 10^4].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Maximum depth is the number of nodes on the longest root-to-leaf path.

An empty tree has depth 0. A single node has depth 1. Right?"

"[3,9,20,null,null,15,7]: longest path is 3->20->15 or 3->20->7, 3 nodes -> 3 ok

[1,null,2]: 1->2, 2 nodes -> 2 ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def height(node):
    if empty:
        return 0
    return 1 + the larger of height(left) and
        height(right)
```

TIME	SPACE
O(n)	O(h)

WHY IT IS SLOW

Linear time, which is optimal; the call stack is as deep as the tree (O(n) when skewed). An iterative BFS counting levels uses a queue instead.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive DFS is already O(n) — let me walk through how depth propagates up."

The recursion computes depth BOTTOM-UP (post-order):

```
depth(null) = 0 <- base case: empty node has depth 0.
depth(node) = 1 + max(depth(node.left),
  depth(node.right))
```

How answers flow upward: each leaf returns 1. Its parent gets max(1,1)+1=2.

That parent's parent gets max(2, sibling_depth)+1, and so on up to the root.

The max() picks the longer branch. The +1 counts this node itself.

Time: O(n) — every node visited once. Space: O(h) — recursion stack depth.

PHASE 4 CLEAN CODE

```python
class _104_MaxDepth:
    def maxDepth(self, root: Optional['TreeNode']) -> int:
        def depth(node):
            if not node:
                return 0
            return max(depth(node.left), depth(node.right)
) + 1
        return depth(root)
```

PHASE 5 TEST & DEBUG

"Let me trace maxDepth on [3,9,20,null,null,15,7]."

```
maxDepth(3): left=maxDepth(9), right=maxDepth(20).
maxDepth(9): left=maxDepth(None)=0,
  right=maxDepth(None)=0 -> 1+max(0,0)=1.
maxDepth(20): left=maxDepth(15)=1, right=maxDepth(7)=1
  -> 1+max(1,1)=2.
maxDepth(3): 1+max(1,2) = 3 ok
```

Call stack (post-order): leaves return 1, their parents return 2, root returns 3.

Edge: root=None -> return 0 ok. Single node -> 1+max(0,0) = 1 ok

WHY 1+max(left,right): depth = this node + the deeper of its two subtrees.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** call stack — O(log n) balanced, O(n) skewed

BFS alternative: count levels with a queue — O(n) time, O(w) space (w=max width).

For very deep trees, BFS avoids recursion stack overflow.

Given more time I'd ask: what about minimum depth? Different — must reach a leaf,

so BFS is actually better there (stops at first leaf found).

IN YOUR APP | Expose | Editorial | Community

EASY | Trees & BST | LeetCode #108 | 4 of 18 | 4/18

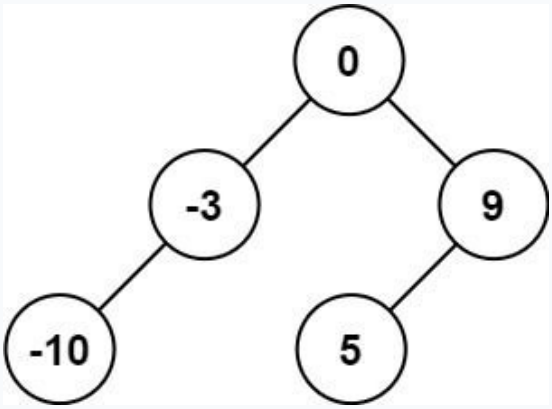
Convert Sorted Array to Binary Search Tree

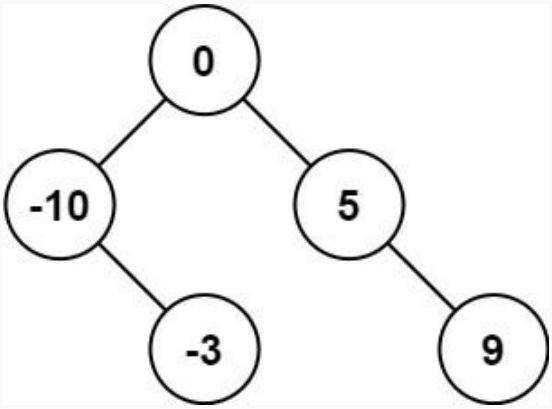
QUESTION

Given an integer array nums where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1

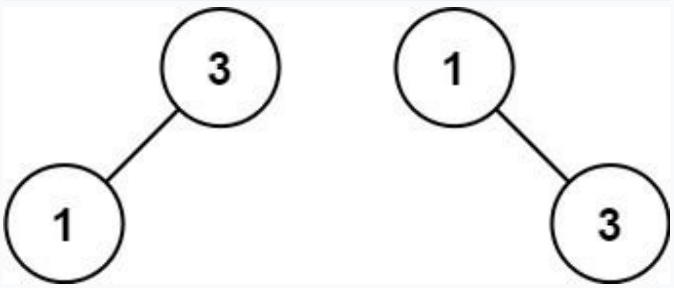
Input	`nums = [-10,-3,0,5,9]`
Output	`[0,-3,9,-10,null,5]`
Explanation	*[0,-10,5,null,-3,null,9] is also accepted:*





Example 2

Input	`nums = [1,3]`
Output	`[3,1]`
Explanation	*[1,null,3] and [3,1] are both height-balanced BSTs.*



CONSTRAINTS

- 1 <= nums.length <= 10^4
- 10^4 <= nums[i] <= 10^4
- nums is sorted in a strictly increasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Convert a sorted array to a height-balanced BST.

Height-balanced means every node's left and right subtree heights differ by at most 1.

Multiple valid answers exist — any height-balanced BST is acceptable. Right?"

"[-10,-3,0,5,9]: pick middle element as root -> 0. Left half [-10,-3] -> left subtree.

Right half [5,9] -> right subtree. Recurse. -> [0,-3,9,-10,null,5] ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def build(nums):
    if empty:
        return nothing
    mid = middle index
    node = nums[mid] with left =
        build(nums[:mid]) and right =
        build(nums[mid + 1:])
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

The slices copy O(n) elements per level, O(n log n) in total. Passing (lo, hi) bounds instead of slicing is O(n) time and O(log n) stack.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Divide and conquer on sorted array — already O(n). Walk-through of how"

the balanced BST is constructed recursively:

`build(lo, hi):`

Base: lo > hi -> return None.

`mid = (lo + hi) // 2 <- middle element becomes root -> guarantees balance.`

`root = TreeNode(nums[mid])`

`root.left = build(lo, mid-1) <- left half becomes left subtree`

`root.right = build(mid+1, hi) <- right half becomes right subtree`

PHASE 3 OPTIMIZE

`return root`

WHY picking the middle guarantees balance: left half has mid-lo elements,

right half has hi-mid elements — differ by at most 1. Both subtrees also

recursively pick their middles -> perfect balance at every level.

Time: O(n). Space: O(log n) recursion (balanced recursion tree).

PHASE 4 CLEAN CODE

```
class _108_SortedArrayToBST:
    def sortedArrayToBST(self, nums: List[int]) -> Optiona
l['TreeNode']:
        def build(left, right):
            if left > right:
                return None
            mid = (left + right) // 2
            return TreeNode(nums[mid], build(left, mid - 1
), build(mid + 1, right))
        return build(0, len(nums) - 1)
```

PHASE 5 TEST & DEBUG

"Let me trace sortedArrayToBST on [-10,-3,0,5,9]."

Mid=2 -> root=0. Left half=[-10,-3], right=[5,9].

Left: mid=1->root=-3. Left=[-10]:mid=0->root=-10(leaf). Right=[]=None.

Right: mid=1->root=9. Left=[5]:mid=0->root=5(leaf). Right=[]=None.

Tree: 0(left=-3(left=-10), right=9(left=5)) ok

In-order: -10,-3,0,5,9 ok (sorted array recovered)

WHY mid as root: puts equal elements in both halves -> height-balanced BST.

Edge: [1] -> mid=0 -> root=1, no children ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(log n)**
every element becomes a node exactly once	for the call stack — tree is balanced by construction

Choosing (l+r)//2 vs (l+r+1)//2 gives slightly different but equally valid trees.

Given more time I'd ask: what if the array is sorted in descending order?

Reverse it first or adjust the index logic — same approach.

EASY | Trees & BST | LeetCode #110 | 5 of 18 | 5/18

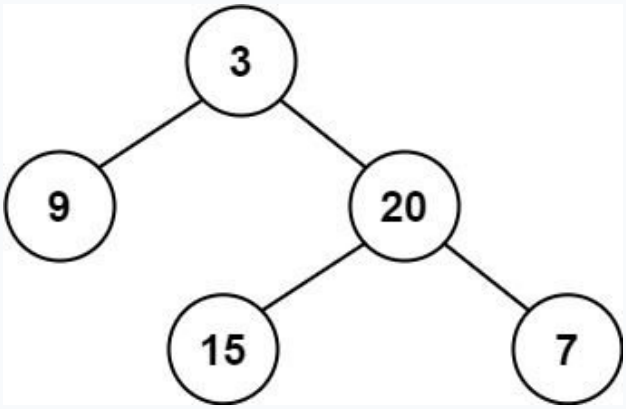
Balanced Binary Tree

QUESTION

Given a binary tree, determine if it is height-balanced.

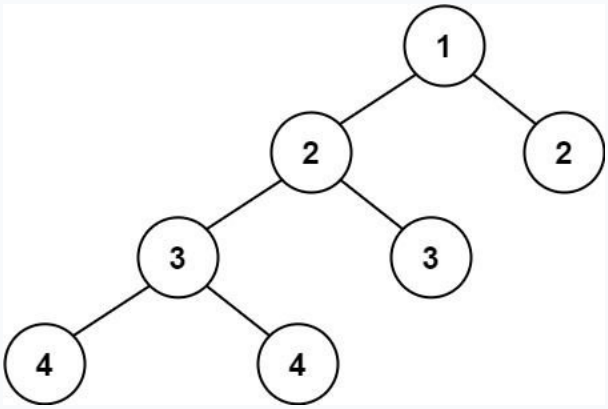
Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`true`



Example 2

Input	`root = [1,2,2,3,3,null,null,4,4]`
Output	`false`



Example 3

Input	`root = []`
Output	`true`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 5000].
- 10^4 <= Node.val <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

A tree is height-balanced if every node's left and right subtree heights

differ by at most 1. Empty tree is balanced. Right?"

"[3,9,20,null,null,15,7]: heights of 9's subtrees: 0 vs 0 ok. 20's: 1 vs 1 ok. 3's: 1 vs 2 ok -> true.

[1,2,2,3,3,null,null,4,4]: left subtree of root has height 3, right has 1 -> false ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def isBalanced(node):
    if empty:
        return True
    if abs(height(left) - height(right)) > 1:
        return False
    return isBalanced(left) and isBalanced(right)
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

height() is recomputed for every node, O(n^2) on a skewed tree. A bottom-up recursion that returns the height (or -1 for unbalanced) visits each node once: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force calls height() repeatedly for the same nodes — O(n²).

I notice we're recomputing heights we've already seen. What if we cache it?

Better: combine the height computation and balance check into one DFS pass.

Return -1 to signal 'unbalanced subtree found' — propagate it up immediately.

Otherwise return the actual height.

A single O(n) pass handles everything.

PHASE 4 CLEAN CODE

```
class _110_BalancedTree:
    def isBalanced(self, root: Optional['TreeNode']) -> bo
ol:
        def check(node):
            if not node:
                return 0
            left  = check(node.left)
            right = check(node.right)
            if left == -1 or right == -1 or abs(left - rig
```

```
ht) > 1:
                return -1        # propagate 'unbalanced'
signal
            return max(left, right) + 1
        return check(root) != -1
```

PHASE 5 TEST & DEBUG

"Let me trace isBalanced on [3,9,20,null,null,15,7] and [1,2,2,3,3,null,null,4,4]."

height(3): h_left=height(9)=1. h_right=height(20)=2. |1-2|=1 <= 1 ok. return 3.

-> True ok

[1,2,2,3,3,null,null,4,4]:

height(4)=1. height(3)=2. height(2): h_left=height(3-with-4)=3, h_right=height(3)=2. |3-2|=1 ok return 4.

height(2) right branch: height(2)=1,height(None)=0->return 2.

height(1): h_left=4, h_right=2. |4-2|=2 > 1 -> return -1 (sentinel for imbalanced).

-> False ok. WHY -1 sentinel: propagates failure without extra global variable.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
single DFS, each node visited once	for the call stack

The -1 sentinel trick is the key interview insight: instead of separate

height and balance checks, fuse them into one return value with a special signal.

Given more time I'd ask: can we rebalance a tree in-place?

Flatten to sorted array (in-order traversal) then rebuild — O(n) time.

EASY | Trees & BST | LeetCode #112 | 6 of 18 | 6/18

Path Sum

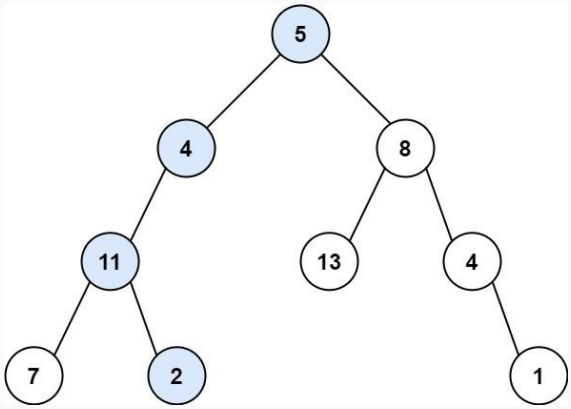
QUESTION

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

A leaf is a node with no children.

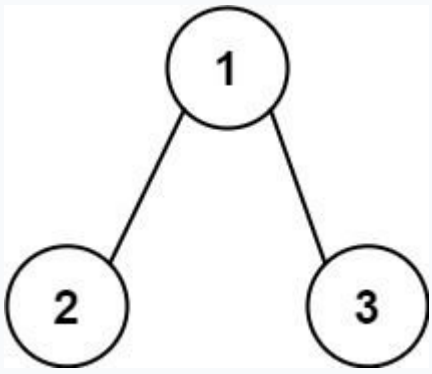
Example 1

Input	`root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22`
Output	`true`
Explanation	*The root-to-leaf path with the target sum is shown.*



Example 2

Input	`root = [1,2,3], targetSum = 5`
Output	`false`
Explanation	*There are two root-to-leaf paths in the tree: (1 --> 2): The sum is 3. (1 --> 3): The sum is 4. There is no root-to-leaf path with sum = 5.*



Example 3

Input	`root = [], targetSum = 0`
Output	`false`
Explanation	*Since the tree is empty, there are no root-to-leaf paths.*

QUESTION

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 5000].
- 1000 <= Node.val <= 1000
- 1000 <= targetSum <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find if any root-to-leaf path sums to targetSum.

A leaf is a node with no children — the path must end at a leaf. Right?

Empty tree -> false (no leaves)."

"[5,4,8,11,null,13,4,7,2,null,null,null,1], target=22:

path 5->4->11->2 = 22 -> true ok

[], target=0 -> false (empty tree has no root-to-leaf path) ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
collect every root-to-leaf path as a list
return True if any path's sum equals targetSum
```

TIME	SPACE
O(n * h)	O(n)

WHY IT IS SLOW

Stores and re-sums every path. Subtracting each node's value from the target while descending and checking at a leaf needs no stored paths: O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Subtraction-as-we-descend is already O(n). Walk-through of the recursion:"

`has_path(node, remaining):`

If node is None -> return False.

`remaining -= node.val <- subtract THIS node's value as we enter it.`

If node is a LEAF (no left, no right):

`return remaining == 0 <- reached a leaf, check if we hit the target exactly.`

`return has_path(left, remaining) or has_path(right, remaining)`

WHY check at leaf only: an interior node with remaining==0 doesn't count — the path

must reach a leaf, not just any node. This catches that edge case cleanly.

PHASE 3 OPTIMIZE

Time: O(n) worst case (unbalanced tree, target not found).
Space: O(h).

PHASE 4 CLEAN CODE

```
class _112_PathSum:
    def hasPathSum(self, root: Optional['TreeNode'], targe
tSum: int) -> bool:
        def hassum(node, remaining):
            if not node:
                return False
            remaining -= node.val
            if not node.left and not node.right:   # leaf
                return remaining == 0
            return hassum(node.left, remaining) or hassum(
node.right, remaining)
        return hassum(root, targetSum)
```

PHASE 5 TEST & DEBUG

"Let me trace hasPathSum on tree [5,4,8,11,null,13,4], targetSum=22."

DFS, passing remaining=targetSum-node.val at each step

root=5: remaining=22-5=17 -> recurse left(4), right(8)

node=4: remaining=17-4=13 -> recurse left(11)

node=11: remaining=13-11=2 -> recurse left(7), right(2)

node=7: leaf, 2-7=-5!=0 -> False

node=2: leaf, 2-2=0 -> True ok -> propagate True up

Call stack: hasPathSum(root,22)->(4,17)->(11,13)->(2,2)->leaf True

WHY DFS: explores each root-to-leaf path completely; short-circuits on first True.

Edge: empty tree -> return targetSum==0 check.

DFS call stack: dfs(root=5,target=22)->dfs(4,17)->dfs(11,13)->dfs(7,6) returns False; dfs(2,11) returns True. visited: each node once. Return True propagates up ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** call stack

The 'leaf check' (no left and no right) is critical — without it, a node with one child that hits 0 would incorrectly return true.

Given more time I'd ask: return all root-to-leaf paths that sum to target?

That's Path Sum II (#113) — collect paths into a result list with backtracking.

EASY | Trees & BST | LeetCode #226 | 7 of 18 | 7/18

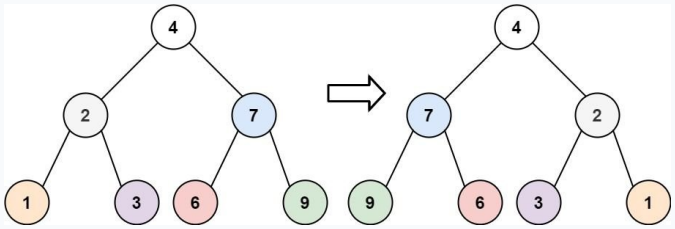
Invert Binary Tree

QUESTION

Given the root of a binary tree, invert the tree, and return its root.

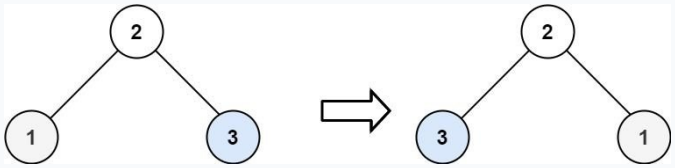
Example 1

Input	`root = [4,2,7,1,3,6,9]`
Output	`[4,7,2,9,6,3,1]`



Example 2

Input	`root = [2,1,3]`
Output	`[2,3,1]`



Example 3

Input	`root = []`
Output	`[]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Invert the tree: swap left and right children at every node recursively.

Return the root. Empty tree -> return None. Right?"

"[4,2,7,1,3,6,9] -> [4,7,2,9,6,3,1]: left and right subtrees are swapped at every level ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def invert(node):
    if empty return
    swap left and right
    invert(left)
    invert(right)
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: every node is touched once. The cost is the recursion stack (O(n) when skewed); an iterative BFS or DFS with an explicit stack has the same bound.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS swap — already O(n). Walk-through of how the inversion propagates:"

`invert(node):`

If node is None -> return None.

`left = invert(node.left) <- recursively invert left subtree first`

`right = invert(node.right) <- recursively invert right subtree first`

`node.left = right <- swap: right becomes left`

`node.right = left <- swap: left becomes right`

`return node`

WHY post-order: we invert the children BEFORE swapping at the current node.

If we swapped first (pre-order), it still works — the key is that BOTH subtrees

get inverted, not just swapped at top level. Order doesn't matter here.

Time: O(n) — visit every node. Space: O(h) — recursion stack.

PHASE 4 CLEAN CODE

```
class _226_InvertTree:
    def invertTree(self, root: Optional['TreeNode']) -> Optional['TreeNode']:
        def invert(node):
            if not node:
                return None
            left  = invert(node.left)
            right = invert(node.right)
            node.left, node.right = right, left
            return node
        return invert(root)
```

PHASE 5 TEST & DEBUG

"Let me trace invertTree on [4,2,7,1,3,6,9]."

`invert(4): swap children -> left=7,right=2. Then recurse:`

`invert(7): swap -> left=9,right=6. invert(9)=leaf,invert(6)=leaf. return 7(9,6).`

PHASE 5 TEST & DEBUG

```
invert(2): swap -> left=3,right=1.
  invert(3)=leaf,invert(1)=leaf. return 2(3,1).
```

Result: 4(left=7(9,6), right=2(3,1)) ok

In-order of original: 1,2,3,4,6,7,9. In-order of inverted: 9,7,6,4,3,2,1 (reversed) ok

WHY post-order (invert children first then swap): order doesn't matter here — swapping at any time works.

Swapping before or after recursion both give the same result.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** call stack

This is the problem Homebrew creator Max Howell famously couldn't solve in a Google interview.

Worth mentioning — it humanises the problem and shows you know your history.

Given more time I'd ask: can we do this iteratively? Yes — BFS or DFS with a stack,

swap children at each node. O(n) time, O(n) space.

IN YOUR APP | Expose | Editorial | Community

EASY | Trees & BST | LeetCode #270 | 8 of 18 | 8/18

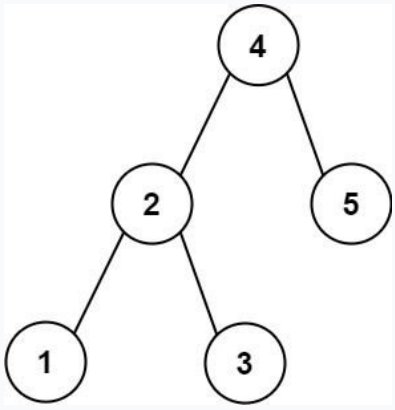
Closest Binary Search Tree Value

QUESTION

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1

Input	`root = [4,2,5,1,3], target = 3.714286`
Output	`4`



Example 2

Input	`root = [1], target = 4.428571`
Output	`1`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 0 <= Node.val <= 10^9
- 10^9 <= target <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the node value in a BST closest to a float target.

If two values are equidistant, return the smaller one. Right?

BST property means we can navigate toward the target rather than scanning all nodes."

"root=[4,2,5,1,3], target=3.714286:

candidates: 4 (dist=0.286), 3 (dist=0.714). 4 is closer -> 4 ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
visit every node of the tree and keep the value
    closest to the target
```

TIME	SPACE
O(n)	O(h)

WHY IT IS SLOW

Ignores the BST ordering. Walk down from the root, going left if the target is smaller and right otherwise, remembering the closest value: O(h) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BST navigation using inequality — O(h). Walk-through:"

`closest = root.val. node = root.`

While node:

If abs(node.val - target) < abs(closest - target): closest = node.val.

If target < node.val: node = node.left.

Elif target > node.val: node = node.right.

Else: return node.val. <- exact match

Return closest.

WHY BST navigation: BST property means as we descend, we move toward target.

Candidates for closest are only along the path from root to where target would be inserted.

WHY update closest at each step: the current node may be closer than any previous node.

Time: O(h). Space: O(1).

PHASE 4 CLEAN CODE

```
class _270_ClosestBST:
    def closestValue(self, root: Optional['TreeNode'], tar
get: float) -> int:
        def close(node, best):
            if not node:
                return best
            if abs(target - node.val) < abs(target - best)
:
                best = node.val
            elif abs(target - node.val) == abs(target - be
st):
                best = min(best, node.val)    # tie -> pic
k smaller
            if target < node.val:
                return close(node.left, best)
            else:
                return close(node.right, best)
        return close(root, root.val)
```

PHASE 5 TEST & DEBUG

"Let me trace closestValue on BST [4,2,5,1,3], target=3.714286."

closest=4. node=4: |4-3.714|=0.286. target<4->go left.

node=2: |2-3.714|=1.714 > 0.286. target>2->go right.

node=3: |3-3.714|=0.714 > 0.286. target>3->go right=None. Stop.

Return 4 ok (4 is closest to 3.714)

target=2.5:

node=4: |4-2.5|=1.5. go left. node=2: |2-2.5|=0.5<1.5->closest=2. go right. node=3: |3-2.5|=0.5.

Tie: closest stays 2 (first encountered). Return 2 ok

WHY BST path: closest is always on the path from root to where target would be inserted.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(h)** for recursion stack. Iterative: O(1) space

BST navigation is the key — we prune entire subtrees we know can't contain

a closer value.

Given more time I'd ask: find the k closest values to target?

That's Closest BST Value II (#272) — use an in-order traversal with a sliding window

or a max-heap of size k.

IN YOUR APP | Expose | Editorial | Community

EASY | Trees & BST | LeetCode #404 | 9 of 18 | 9/18

Sum of Left Leaves

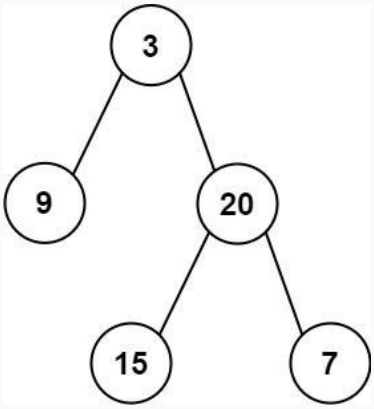
QUESTION

Given the root of a binary tree, return the sum of all left leaves.

A leaf is a node with no children. A left leaf is a leaf that is the left child of another node.

Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`24`
Explanation	*There are two left leaves in the binary tree, with values 9 and 15 respectively.*



Example 2

Input	`root = [1]`
Output	`0`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 1000].
- 1000 <= Node.val <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the sum of all left leaf node values. A left leaf is a leaf node that is the left child. Right?"

"A few quick questions:

If root is a leaf, it's NOT a left leaf (it has no parent)? Correct.

Empty tree -> 0?"

"Let me walk: root=[3,9,20,null,null,15,7] -> left leaves: 9,15. sum=24 ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
visit every node
if a node is the left child of its parent and has no
    children:
    add its value
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: every node is visited once. The cost is the recursion stack, O(n) for a skewed tree; an iterative stack or BFS does the same work.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS with is_left flag — already O(n). Walk-through:"

`dfs(node, is_left):`

If node is None -> return 0.

If node is a LEAF (no children) and is_left is True -> return node.val.

`return dfs(node.left, True) + dfs(node.right, False)`

HOW the flag propagates: the left child of any node is called with is_left=True.

The right child is called with is_left=False.

A leaf that received is_left=True is a LEFT leaf -> add its value.

A leaf that received is_left=False is a RIGHT leaf -> don't add.

WHY we need the leaf check: if we counted ALL left-side nodes (not just leaves),

we'd include interior nodes that happen to be left children.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _404_SumOfLeftLeaves:
    def sumOfLeftLeaves(self, root):
        def dfs(node, is_left_child):
            if not node:
                return 0
            if not node.left and not node.right:
                return node.val if is_left_child else 0
            return dfs(node.left, True) + dfs(node.right,
False)
        return dfs(root, False)
```

PHASE 5 TEST & DEBUG

"Let me trace sumOfLeftLeaves on [3,9,20,null,null,15,7]."

```
dfs(3, is_left=False): dfs(9, is_left=True): 9 is leaf
  AND is_left -> sum+=9.
dfs(20, is_left=False): dfs(15, is_left=True): 15 is
  leaf, is_left -> sum+=15.
dfs(7, is_left=False): 7 is leaf, NOT is_left -> don't
  add. return 0.
```

Total: 9+15=24 ok

WHY pass is_left flag: we need to know if current node is a LEFT child.

A leaf is only summed if it's reached via a left edge from its parent.

Edge: no left leaves -> return 0 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Follow-up: sum of right leaves — flip the flag logic.

Follow-up: sum of all leaves — remove the is_left_child check.

EASY | Trees & BST | LeetCode #501 | 10 of 18 | 10/18

Find Mode in Binary Search Tree

QUESTION

Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

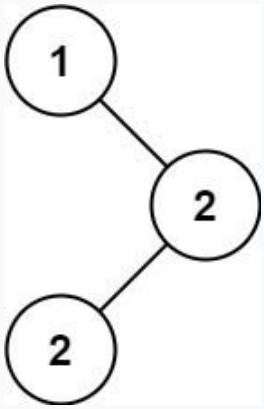
Assume a BST is defined as follows:

CONSTRAINTS

- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1

Input	`root = [1,null,2,2]`
Output	`[2]`



Example 2

Input	`root = [0]`
Output	`[0]`

- The number of nodes in the tree is in the range [1, 10^4].
- 10^5 <= Node.val <= 10^5
- Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return all values that appear the most frequently in the BST.

This BST allows duplicates (left <= node, right >= node).

Return all modes — there can be multiple. Any order is fine. Right?

Follow-up: can we do it without extra space (O(1) beyond recursion stack)?"

"[1,null,2,2]: 2 appears twice, 1 once -> mode is [2] ok

PHASE 1 CLARIFY

[0]: only element -> mode is [0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = a hash map of how many times each value
    occurs (full traversal)
return the values with the highest count
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear, but the map stores every value. In-order traversal visits equal values together, so a running count and the previous value find the mode in O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (O(1) space using in-order traversal)

BST in-order traversal visits nodes in sorted order.

Duplicates in a BST are adjacent in in-order — so we can count runs without a map.

Track: current value, current run count, max count seen, result list.

When current run count exceeds max, reset result. When equal, append.

Use nonlocal to mutate state inside the inner helper."

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```python
class _501_FindMode:
    def findMode(self, root: Optional['TreeNode']) -> List
[int]:
        modes: List[int] = []
        cur_val  = [None]
        cur_cnt  = [0]
        max_cnt  = [0]

        def inorder(node):
            if not node:
                return
            inorder(node.left)
            if node.val == cur_val[0]:
                cur_cnt[0] += 1
            else:
                cur_val[0] = node.val
                cur_cnt[0] = 1
            if cur_cnt[0] > max_cnt[0]:
                max_cnt[0] = cur_cnt[0]
                modes.clear()
                modes.append(node.val)
            elif cur_cnt[0] == max_cnt[0]:
                modes.append(node.val)
            inorder(node.right)

        inorder(root)
        return modes
```

PHASE 5 TEST & DEBUG

"Let me trace findMode on BST [1,null,2,2]."

In-order traversal: 1, 2, 2.

Visit 1: curr_val=1,curr_count=1,max_count=1. modes=[1].

Visit 2: curr_val!=1->reset: curr_val=2,curr_count=1. 1==max_count->modes=[1,2].

Visit 2: curr_val==2->curr_count=2. 2>max_count=1->max_count=2,modes=[2].

Return [2] ok

WHY in-order: BST in-order = sorted -> duplicates are adjacent -> easy consecutive counting.

WHY reset modes when new max: previous modes are superseded by more frequent element.

Edge: all unique [1,2,3]->modes=[1,2,3] (all appear once, all tied for max_count=1) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

Brute force: O(n) time, O(n) space.

O(1) space version: O(n) time, O(1) extra space (modes list not counted).

The in-order run-counting trick works because BST duplicates are always adjacent

in sorted traversal — a key property of this BST variant.

Given more time I'd ask: what if it were a regular BST (no duplicates)?

The mode would always be the single element — trivially the root if n=1.

EASY | Trees & BST | LeetCode #530 | 11 of 18 | 11/18

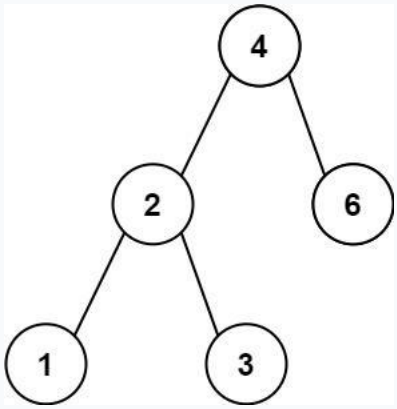
Minimum Absolute Difference in BST

QUESTION

Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree.

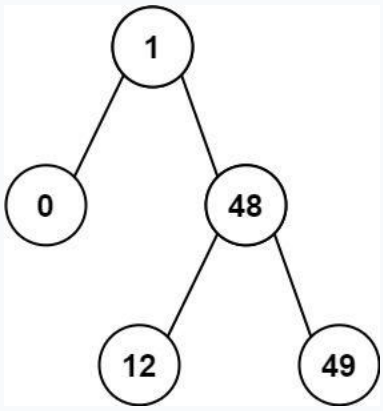
Example 1

Input	`root = [4,2,6,1,3]`
Output	`1`



Example 2

Input	`root = [1,0,48,null,null,12,49]`
Output	`1`



CONSTRAINTS

- The number of nodes in the tree is in the range [2, 10^4].
- 0 <= Node.val <= 10^5
- Note: This question is the same as 783: https://leetcode.com/problems/minimum-distance-between-bst-nodes/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the minimum absolute difference between any two nodes in a BST.

PHASE 1 CLARIFY

Inorder traversal gives sorted values; minimum diff is between consecutive values. Right?"

"A few quick questions:

At least 2 nodes guaranteed? Yes."

"Let me walk: root=[4,2,6,1,3] -> inorder=[1,2,3,4,6]. min diffs: 1,1,1,2 -> 1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
values = every node's value
for pair in pairs:
    best = min(best, |a - b|)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Compares all pairs. In-order traversal visits values in sorted order, so only neighbours need comparing: O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"In-order with previous node tracking — already O(n). Walk-through:"

In-order gives BST nodes in sorted ascending order.

`prev = None initially.`

`inorder(node):`

`inorder(node.left) <- go left first to reach smallest values`

`if prev is not None:`

`min_diff = min(min_diff, node.val - prev.val)`

<- consecutive in-order nodes have minimum possible gap in a BST.

`prev = node <- update: this node is 'previous' for the next in-order node`

`inorder(node.right) <- then visit right subtree`

WHY comparing only consecutive in-order nodes is sufficient:

The minimum absolute difference between ANY two BST nodes must occur between

adjacent values in sorted order — never between non-adjacent ones.

Time: O(n). Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _530_MinAbsoluteDifferenceInBST:
    def getMinimumDifference(self, root):
        self_min_diff = [float('inf')]
        self_prev = [None]
        def inorder(node):
            if not node: return
            inorder(node.left)
            if self_prev[0] is not None:
                self_min_diff[0] = min(self_min_diff[0], n
ode.val - self_prev[0])
            self_prev[0] = node.val
            inorder(node.right)
        inorder(root)
        return self_min_diff[0]
```

PHASE 5 TEST & DEBUG

"Let me trace getMinimumDifference on BST [4,2,6,1,3]."

In-order: 1, 2, 3, 4, 6.

Visit 1: prev=None->set prev=1.

Visit 2: diff=2-1=1. min_diff=1. prev=2.

Visit 3: diff=3-2=1. min_diff=1. prev=3.

Visit 4: diff=4-3=1. min_diff=1. prev=4.

Visit 6: diff=6-4=2. min_diff stays 1. prev=6.

Return 1 ok

WHY in-order: BST in-order = sorted ascending -> consecutive elements have minimum difference.

Min difference in BST always occurs between adjacent nodes in sorted order.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

Same approach as Minimum Distance Between BST Nodes (#783) — identical problem.

Follow-up: kth smallest difference — collect all diffs, sort, return kth.

Follow-up: if tree is not a BST, collect all values and sort first.

EASY | Trees & BST | LeetCode #543 | 12 of 18 | 12/18

Diameter of Binary Tree

QUESTION

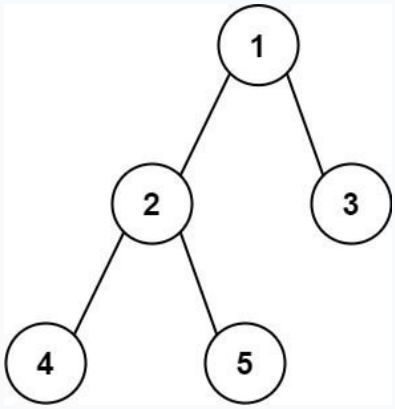
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1

Input	`root = [1,2,3,4,5]`
Output	`3`
Explanation	*3 is the length of the path [4,2,1,3] or [5,2,1,3].*



Example 2

Input	`root = [1,2]`
Output	`1`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Diameter = length of the longest path between any two nodes (measured in edges).

The path may or may not pass through the root. Right?"

"A few quick questions:

Single node -> diameter 0? Yes. Two nodes -> diameter 1? Yes."

"Let me walk: root=[1,2,3,4,5] -> path 4->2->1->3 has 3 edges -> diameter=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    leftHeight = height(left) and rightHeight =
        height(right), each computed by a fresh
        traversal
    diameter = the largest leftHeight + rightHeight
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

Recomputes heights for every node, O(n^2) on a skewed tree. One post-order pass that returns the height and updates the best left + right on the way up is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Post-order DFS: diameter through each node = left_depth + right_depth — O(n). Walk-through:"

`def depth(node):`

If node is None: return 0.

`left = depth(node.left). right = depth(node.right).`

`ans = max(ans, left + right). <- diameter through this node`

`return 1 + max(left, right). <- max depth for parent's calculation`

WHY left + right: the diameter using this node as 'bend' = longest path from left subtree

through this node to right subtree = left_depth + right_depth.

WHY return 1 + max(left, right): for the parent, this node contributes its own height

(the longer of its two subtrees + 1 for itself).

Global max: maintained across all nodes.

Time: O(n). Space: O(h) call stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _543_DiameterOfBinaryTree:
    def diameterOfBinaryTree(self, root):
        best = [0]
        def depth(node):
            if not node:
                return 0
            left = depth(node.left)
            right = depth(node.right)
            best[0] = max(best[0], left + right)
            return 1 + max(left, right)
        depth(root)
        return best[0]
```

PHASE 5 TEST & DEBUG

"Let me trace diameterOfBinaryTree on [1,2,3,4,5]."

```
depth(4): L=0, R=0. diameter candidate=0+0=0. return
  1.
depth(5): same -> return 1. diameter=0.
depth(2): L=depth(4)=1, R=depth(5)=1. candidate=1+1=2.
  ans=2. return 1+max(1,1)=2.
depth(3): L=0, R=0. candidate=0. return 1.
depth(1): L=depth(2)=2, R=depth(3)=1. candidate=2+1=3.
  ans=3. return 1+max(2,1)=3.
```

Answer: 3 ok (path 4->2->5->? No: 4->2->1->3 has length 3 edges)

Actually path is 4->2->5 or 4->2->1->3 — both length 3. ans=max=3 ok

WHY left+right at each node: diameter passing through this node = left depth + right depth.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
single post-order DFS	call stack

The single-element list best=[0] is the Python pattern for mutating a variable in a closure.

Follow-up: Binary Tree Maximum Path Sum (#124) — same pattern with sum instead of count.

Follow-up: Diameter of N-Ary Tree (#1522) — extend to multiple children.

EASY | Trees & BST | LeetCode #572 | 13 of 18 | 13/18

Subtree of Another Tree

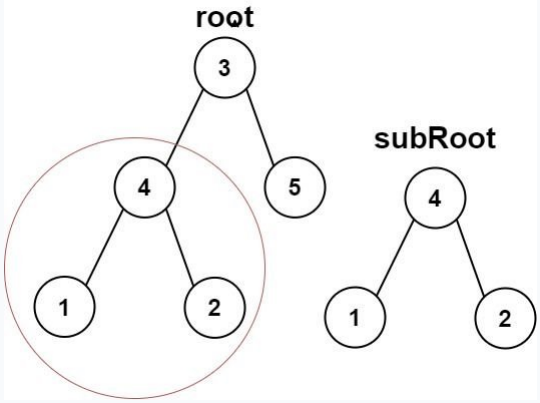
QUESTION

Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the same structure and node values of subRoot and false otherwise.

A subtree of a binary tree tree is a tree that consists of a node in tree and all of this node's descendants. The tree tree could also be considered as a subtree of itself.

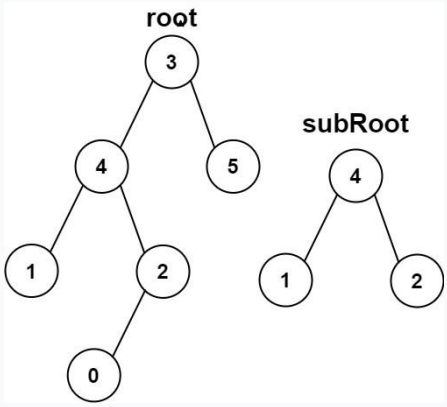
Example 1

Input	`root = [3,4,5,1,2], subRoot = [4,1,2]`
Output	`true`



Example 2

Input	`root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]`
Output	`false`



CONSTRAINTS

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- 10^4 <= root.val <= 10^4
- 10^4 <= subRoot.val <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return true if subRoot appears as a subtree anywhere in root.

A subtree means a node and ALL its descendants match — not just a portion.

The tree itself is a subtree of itself. Right?"

"root=[3,4,5,1,2], subRoot=[4,1,2]: node 4 in root has left=1,right=2 -> matches ok

root=[3,4,5,1,2,null,null,null,null,0], subRoot=[4,1,2]:

node 4 in root has extra child 0 -> doesn't match ok"

Any constraints on tree height or node count I should design around?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in main_tree:
    check whether the tree rooted there is
        identical to subRoot
return True if any is
```

TIME	SPACE
O(m * n)	O(h)

WHY IT IS SLOW

Compares from every node (m main nodes, n sub nodes). Serialise both trees (with null markers) and test whether one string is inside the other with KMP: O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS match root-to-root, then recurse — O(mn). Walk-through:"

`def isSame(s, t):`

If both None: True. If one None: False. If vals differ: False.

`return isSame(s.left, t.left) AND isSame(s.right, t.right).`

`def isSubtree(root, subRoot):`

If root is None: False.

If isSame(root, subRoot): True.

`return isSubtree(root.left, subRoot) OR isSubtree(root.right, subRoot).`

WHY try every node in root: subRoot can be rooted at any node in root.

HOW it works: for each node in root, check if the subtree rooted there is identical to subRoot.

isSame is the same as Q100 — returns True only if structures AND values match exactly.

Time: O(mn) where m=|root|, n=|subRoot|. Space: O(h)."

Time: O(n) — optimal single-pass solution. Linear time is best achievable.

PHASE 4 CLEAN CODE

```
class _572_SubtreeAnotherTree:
    def isSubtree(self, root: Optional['TreeNode'], subRoo
t: Optional['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and same(p.left, q.left)
 and same(p.right, q.right)

        def subtree(node, sub):
            if not node:
                return False
            if same(node, sub):
                return True
            return subtree(node.left, sub) or subtree(node
.right, sub)

        return subtree(root, subRoot)
```

PHASE 5 TEST & DEBUG

"Let me trace isSubtree on root=[3,4,5,1,2], subRoot=[4,1,2]."

DFS: check if any node in root matches subRoot's root (val=4)

root=3: 3!=4 -> recurse left(4), right(5)

node=4: 4==4 -> run isSameTree(node,subRoot)

isSameTree: 4==4 ok, left: 1==1 ok, right: 2==2 ok -> True

Return True ok

isSameTree handles null checks: both null->True, one null->False, different val->False.

WHY separate isSameTree: modular; called once per matching-root candidate.

Time O(m*n) worst case (every node matches root, check full subRoot).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(h_root + h_sub)**
at each of m nodes we may run same() which takes O(n)	for call stacks

Optimisation: serialise both trees and use string matching (KMP or Rabin-Karp)

for O(m+n) time — worth mentioning as an advanced follow-up.

Be careful with serialisation: '[1,2]' would incorrectly match '[12]' without

delimiters like '#' for null and ',' between values.

Given more time I'd ask: what if we query many different subRoots against

the same root? Pre-serialise root once and use string search for each query.

IN YOUR APP | Expose | Editorial | Community

EASY | Trees & BST | LeetCode #637 | 14 of 18 | 14/18

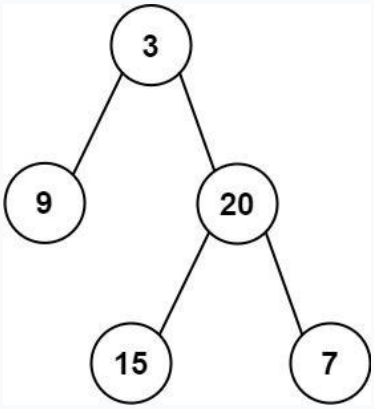
Average of Levels in Binary Tree

QUESTION

Given the root of a binary tree, return the average value of the nodes on each level in the form of an array. Answers within 10^{-5} of the actual answer will be accepted.

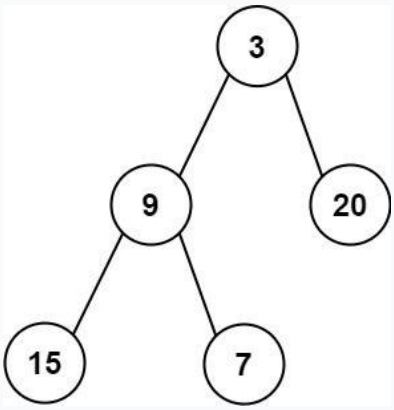
Example 1

Input	`root = [3,9,20,null,null,15,7]`
Output	`[3.00000,14.50000,11.00000]`
Explanation	*The average value of nodes on level 0 is 3, on level 1 is 14.5, and on level 2 is 11. Hence return [3, 14.5, 11].*



Example 2

Input	`root = [3,9,20,15,7]`
Output	`[3.00000,14.50000,11.00000]`



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 2^{31} <= Node.val <= 2^{31} - 1

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the average value of nodes at each level of a binary tree.

Result has one average per level. Right?"

"A few quick questions:

Return as a list of floats? Yes. Empty tree -> empty list? Yes."

"Let me walk: root=[3,9,20,null,null,15,7] -> [3.0, 14.5, 11.0] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for d in ds:
    DFS the whole tree collecting the values at
        depth d
    average them
```

TIME	SPACE
O(n * h)	O(n)

WHY IT IS SLOW

Rescans the tree once per level. A BFS that processes one level per round, summing and counting, visits each node once: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BFS level-snapshot — same as #102, computing avg per level. Already O(n)."

Walk-through:

For each BFS level snapshot:

Pop exactly snapshot_size nodes.

Sum their values.

Compute average = total / snapshot_size.

Append to result.

Enqueue all non-null children.

WHY divide by snapshot_size (not queue size): queue may have grown with children

by the time we divide — use the count we saved BEFORE processing this level.

Time: O(n). Space: O(w) where w = max tree width.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _637_AverageOfLevels:
    def averageOfLevels(self, root):
        if not root:
            return []
```

```
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            level_sum = 0
            for _ in range(level_size):
                node = queue.popleft()
                level_sum += node.val
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
            result.append(level_sum / level_size)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace averageOfLevels on [3,9,20,null,null,15,7]."

Level 0: [3] -> avg=3.0. Enqueue 9,20.

Level 1: [9,20] -> sum=29, count=2, avg=14.5. Enqueue 15,7.

Level 2: [15,7] -> sum=22, count=2, avg=11.0.

Result: [3.0, 14.5, 11.0] ok

WHY snapshot queue size: same as Q102 — isolates each level for separate averaging.

WHY float division: levels may have non-integer averages.

Edge: single root [5] -> [[5.0]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(w)**
visit every node once	max queue size = max tree width

Follow-up: Level Order Traversal (#102) — collect values per level (no averaging).

Follow-up: if tree is very wide, BFS space could be O(n) — DFS with level tracking is O(h).

EASY | Trees & BST | LeetCode #653 | 15 of 18 | 15/18

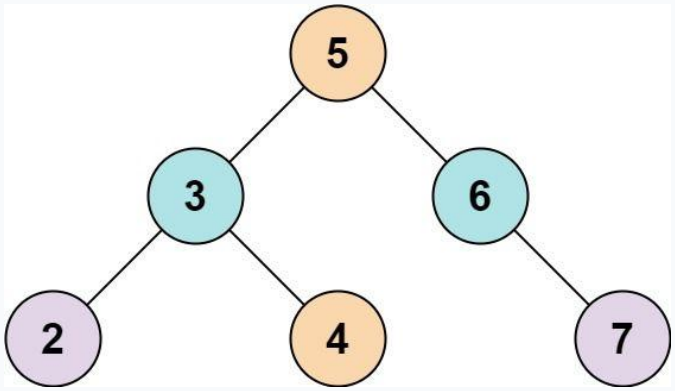
Two Sum IV - Input is a BST

QUESTION

Given the root of a binary search tree and an integer k, return true if there exist two elements in the BST such that their sum is equal to k, or false otherwise.

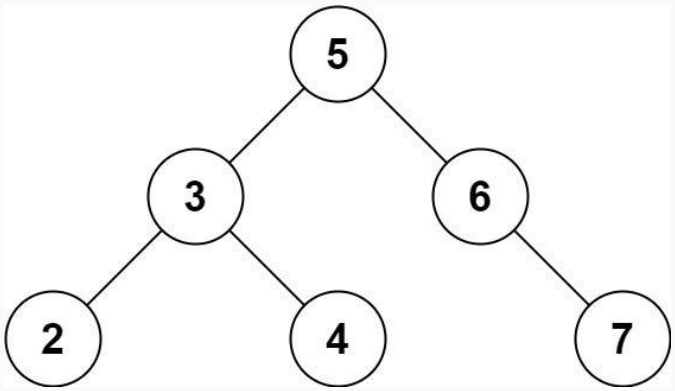
Example 1

Input	`root = [5,3,6,2,4,null,7], k = 9`
Output	`true`



Example 2

Input	`root = [5,3,6,2,4,null,7], k = 28`
Output	`false`



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 10^4].
- 10^4 <= Node.val <= 10^4
- root is guaranteed to be a valid binary search tree.
- 10^5 <= k <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find two nodes in a BST that sum to k. Return true if such a pair exists. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Can the same node be used twice? No — must be two different nodes.

Works on any BST structure? Yes."

"Let me walk: root=[5,3,6,2,4,null,7], k=9 -> 2+7=9 ok -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for node in nodes:
    search the whole tree for the value k -
        node.val (a different node)
return True if found
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

A full search per node. Keep a hash set of values seen during one traversal (O(n) time, O(n) space), or run two pointers over the in-order values (or two BST iterators) in O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra set.

BST iterator two-pointer: one ascending (lo), one descending (hi).

This achieves O(n) time with O(h) space for iterator stacks.

Simpler: DFS + set is already O(n) time and O(n) space.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _653_TwoSumIVInputIsABST:
    def findTarget(self, root, k):
        seen = set()
        def dfs(node):
            if not node:
                return False
            if k - node.val in seen:
                return True
            seen.add(node.val)
            return dfs(node.left) or dfs(node.right)
        return dfs(root)
```

PHASE 5 TEST & DEBUG

"Let me trace twoSumBST on BST [5,3,6,2,4,null,7], target=9."

Inorder traversal gives sorted array [2,3,4,5,6,7]. Two pointer on this list:

lo=0(val=2), hi=5(val=7). 2+7=9==target -> found ok

PHASE 5 TEST & DEBUG

target=28: 2+7=9<28->lo++. 3+7=10<28->lo++. ... 6+7=13<28->lo catches hi -> not found -> False ok

Alternative: BST with hash set.

In-order, for each node val: if (target-val) in seen -> True. seen.add(val).

val=2: target-2=7 not in {}. val=3: 6 not in {2}. ... val=6: 3 in {2,3,4,5,6}? No->6 in set->...

At val=7: 9-7=2 in {2,3,4,5,6} -> True ok

WHY tree details: return values from children drive decisions at the parent (post-order), so base cases must be correct.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** set + O(h) stack

Two Sum IV using BST iterators achieves O(h) space — same as #1214 Two Sum BSTs.

Follow-up: find the actual pair — track which values we added.

Follow-up: k-Sum in BST — generalise with backtracking.

EASY | Trees & BST | LeetCode #700 | 16 of 18 | 16/18

Search in a Binary Search Tree

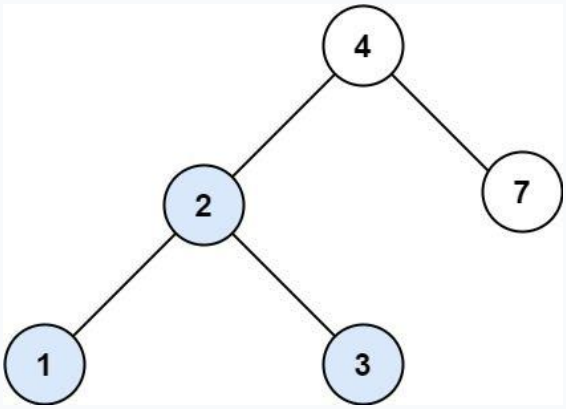
QUESTION

You are given the root of a binary search tree (BST) and an integer val.

Find the node in the BST that the node's value equals val and return the subtree rooted with that node. If such a node does not exist, return null.

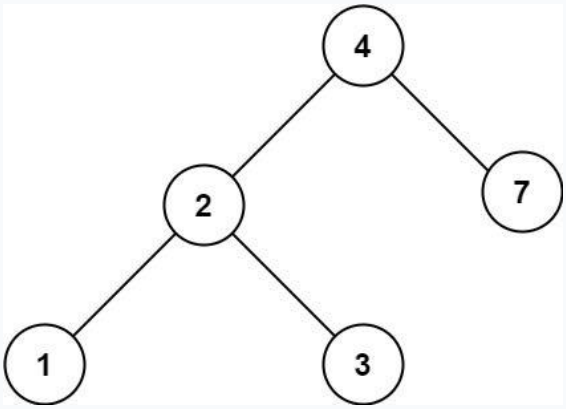
Example 1

Input	`root = [4,2,7,1,3], val = 2`
Output	`[2,1,3]`



Example 2

Input	`root = [4,2,7,1,3], val = 5`
Output	`[]`



CONSTRAINTS

- The number of nodes in the tree is in the range [1, 5000].
- $1 <= Node.val <= 10^7$
- root is a binary search tree.
- $1 <= val <= 10^7$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Search for a value in a BST. Return the subtree rooted at that node, or null if not found. Right?"

"A few quick questions:

Return the node itself (not just the value)? Yes.

BST property: left < node < right? Yes."

"Let me walk: root=[4,2,7,1,3], val=2 -> return node(2) (subtree [2,1,3]) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
visit every node of the tree in any order and return
    the one with the target value
```

TIME	SPACE
O(n)	O(h)

WHY IT IS SLOW

Ignores the BST ordering. Go left if the target is smaller and right if larger: O(h) time and O(1) space (iterative).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BST search — O(h), vastly better than O(n) linear scan. Walk-through:"

`search(node, val):`

If node is None -> return None (not found).

If val == node.val -> return node (found!).

If val < node.val -> return search(node.left, val)

<- BST property: if target is smaller, it must be in left subtree.

`return search(node.right, val)`

<- target is larger -> must be in right subtree.

WHY O(h) not O(n): at EACH node we eliminate one entire subtree from the search.

Balanced BST: O(log n). Worst-case skewed: O(n).

Time: O(h). Space: O(h) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _700_SearchInBST:
    def searchBST(self, root, val):
        while root:
            if val == root.val:
                return root
            elif val < root.val:
```

```
            root = root.left
        else:
            root = root.right
    return None
```

PHASE 5 TEST & DEBUG

"Let me trace searchBST on [4,2,7,1,3], val=2."

node=4: val=2 < 4 -> go left.

node=2: val=2 == 2 -> return node(2,left=1,right=3) ok

val=5:

node=4: 5>4 -> go right. node=7: 5<7 -> go left. node=None -> return None ok

WHY BST property: val < node -> target in left subtree. val > node -> right subtree.

O(h) not O(n): we eliminate half the remaining search space (BST guarantee) at each step.

Edge: val == root.val -> return root immediately ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(h)	**O(1)**
O(log n) balanced, O(n) worst skewed	iterative

Follow-up: Insert into BST (#701) — search for insertion point, create node.

Follow-up: Delete Node in BST (#450) — find then handle 3 cases (leaf, one child, two children).

EASY | Trees & BST | LeetCode #783 | 17 of 18 | 17/18

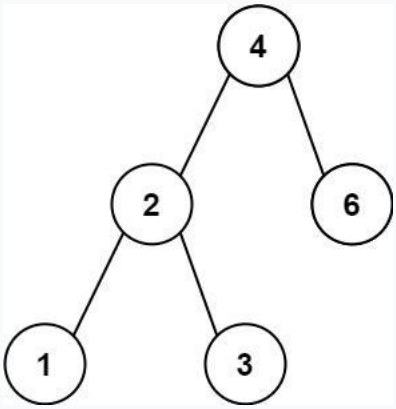
Minimum Distance Between BST Nodes

QUESTION

Given the root of a Binary Search Tree (BST), return the minimum difference between the values of any two different nodes in the tree.

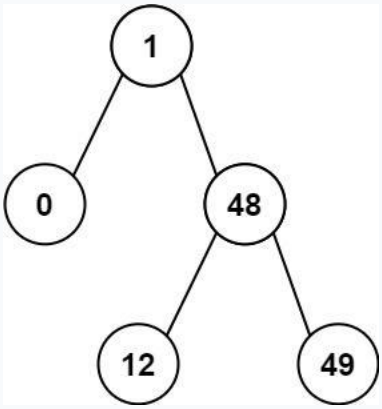
Example 1

Input	`root = [4,2,6,1,3]`
Output	`1`



Example 2

Input	`root = [1,0,48,null,null,12,49]`
Output	`1`



CONSTRAINTS

- The number of nodes in the tree is in the range [2, 100].
- 0 <= Node.val <= 10^5
- Note: This question is the same as 530: https://leetcode.com/problems/minimum-absolute-difference-in-bst/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the minimum difference between any two different nodes in a BST.

Identical to Minimum Absolute Difference (#530). Right?"

PHASE 1 CLARIFY

"A few quick questions:

Same problem as #530 with a slightly different name? Yes — same solution applies."

"Let me walk: root=[4,2,6,1,3] -> inorder=[1,2,3,4,6], min diffs=1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(nodes):
    best = min(best, |a - b|)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Compares all pairs. In-order traversal visits values in sorted order, so only neighbours matter: O(n) time and O(h) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"In-order traversal collecting values, find min difference — O(n). Walk-through:"

In-order traversal of BST yields values in sorted ascending order.

While traversing: track previous value (prev). min_diff = min(min_diff, curr.val - prev).

Morris traversal or recursive:

`def inorder(node):`

If None: return.

`inorder(node.left). process(node.val). inorder(node.right).`

WHY in-order: BST in-order = sorted -> adjacent elements in traversal are closest in value.

Min difference between any two BST nodes = min diff between consecutive in-order values.

Track prev value, compute diff at each step.

Time: O(n). Space: O(h) recursion.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _783_MinDistanceBSTNodes:
    def minDiffInBST(self, root):
        self_min = [float('inf')]
        self_prev = [None]
        def inorder(node):
            if not node: return
            inorder(node.left)
            if self_prev[0] is not None:
```

```
                self_min[0] = min(self_min[0], node.val -
self_prev[0])
            self_prev[0] = node.val
            inorder(node.right)
        inorder(root)
        return self_min[0]
```

PHASE 5 TEST & DEBUG

"Let me trace getMinimumDifference on BST [4,2,6,1,3]."

In-order traversal gives: 1, 2, 3, 4, 6.

prev=None. Visit 1: prev=1.

Visit 2: diff=2-1=1. min_diff=1. prev=2.

Visit 3: diff=3-2=1. min_diff=1. prev=3.

Visit 4: diff=4-3=1. min_diff=1. prev=4.

Visit 6: diff=6-4=2 > 1. min_diff=1. prev=6.

Return 1 ok

Same as Q530 (Minimum Absolute Difference in BST).

WHY in-order: sorted order -> adjacent elements -> minimum gap always between consecutive nodes.

Edge: [1,0,2] -> in-order: 0,1,2 -> diffs: 1,1 -> min=1 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(h)

This is the same as Minimum Absolute Difference (#530) — both problems are identical.

Follow-up: if values can repeat, use <= instead of < in comparison.

Follow-up: k smallest differences — collect all diffs into array, sort, return kth.

EASY | Trees & BST | LeetCode #938 | 18 of 18 | 18/18

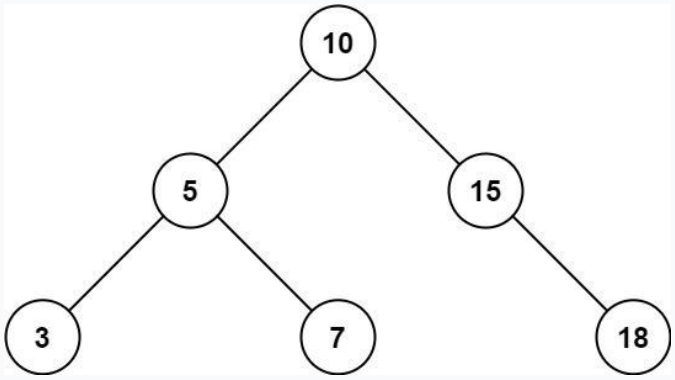
Range Sum of BST

QUESTION

Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

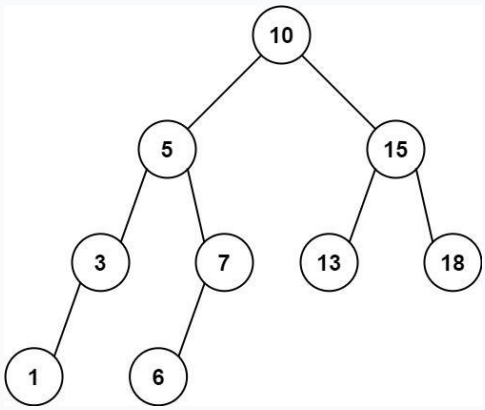
Example 1

Input	`root = [10,5,15,3,7,null,18], low = 7, high = 15`
Output	`32`
Explanation	*Nodes 7, 10, and 15 are in the range [7, 15]. 7 + 10 + 15 = 32.*



Example 2

Input	`root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10`
Output	`23`
Explanation	*Nodes 6, 7, and 10 are in the range [6, 10]. 6 + 7 + 10 = 23.*



CONSTRAINTS

- The number of nodes in the tree is in the range [1, $2 * 10^4$].
- $1 <= Node.val <= 10^5$
- $1 <= low <= high <= 10^5$
- All Node.val are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the sum of values of all BST nodes whose value is in [low, high] inclusive. Right?"

"A few quick questions:

Use BST property to prune? Yes — no need to visit subtrees outside the range."

"Let me walk: root=[10,5,15,3,7,null,18], low=7, high=15 -> 7+10+15=32 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
visit every node of the tree and add the values
    inside [low, high]
```

TIME	SPACE
O(n)	O(h)

WHY IT IS SLOW

Ignores the BST ordering. Skip the left subtree when a node is below low and the right subtree when it is above high: O(h + matching nodes).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"BST range-pruning DFS — O(n) worst case but much faster in practice. Walk-through:"

`range_sum(node, low, high):`

If node is None -> return 0.

`total = 0`

`if low <= node.val <= high: total += node.val <- this node is in range`

`if node.val > low: total += range_sum(node.left, low, high)`

<- left subtree might have nodes >= low; visit it.

<- if node.val <= low, entire left subtree is also <= low -> skip (pruning!).

`if node.val < high: total += range_sum(node.right, low, high)`

<- right subtree might have nodes <= high; visit it.

<- if node.val >= high, entire right subtree is also >= high -> skip.

WHY BST pruning is correct: BST property guarantees ALL left children < node.val

and ALL right children > node.val, so we can skip entire subtrees confidently.

Time: O(n) worst case, much less when range is narrow. Space: O(h).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _938_RangeSumBST:
    def rangeSumBST(self, root, low, high):
        if not root:
            return 0
        total = root.val if low <= root.val <= high else 0
        if root.val > low:
            total += self.rangeSumBST(root.left, low, high
)
        if root.val < high:
            total += self.rangeSumBST(root.right, low, hig
h)
        return total
```

PHASE 5 TEST & DEBUG

"Let me trace rangeSumBST on [10,5,15,3,7,null,18], low=7,high=15."

dfs(10): 7<=10<=15 -> sum+=10=10.

dfs(5): 5<7 -> don't add. 5<=7->might have right child in range -> dfs(5.right=7):

7>=7 and 7<=15 -> sum+=7=17. No children in range (7.left=6<7, skip; 7.right=None).

dfs(15): 15>=7 and 15<=15 -> sum+=15=32.
dfs(15.left=None). dfs(15.right=18): 18>15->skip (but might still have left child >= low... dfs(18.left=None)).

Total: 32 ok

WHY still recurse even when node out of range: subtree might contain in-range nodes.

WHY BST property prunes: if node.val < low, skip LEFT subtree; if > high, skip RIGHT.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) worst, O(log n + k) average where k = nodes in range	**O(h)**

Follow-up: Count BST nodes in range — same structure, count instead of sum.

Follow-up: Kth Smallest in BST (#230) — use inorder traversal with counter.

IN YOUR APP | Expose | Editorial | Community